NORMATIVE IMPLEMENTATION DIRECTIVE

# PLC Engineering Simulator

## Codex Master Implementation Directive

*Phase 1 of 4: Product Constitution, Safety Wall, Architecture, and Governance*

PROFESSIONAL | BRAND-NEUTRAL | EDUCATIONAL | OFFLINE

**VirtualUniverse has no adapter to PhysicalUniverse.**

The product shall simulate engineering decisions and consequences with high training-transfer fidelity while remaining permanently incapable of communicating with or operating physical industrial equipment.

| | |
|---|---|
| **Status** | Phase 1 authored; Phases 2-4 reserved |
| **Owner** | Scott |
| **Research baseline** | Govs PLC project.md \| SHA-256 f05c0832...d3fbf55 |
| **Revision** | 0.1 \| August 27, 2026 |

CONTROLLING PRODUCT TRUTH: Build a professional, brand-neutral PLC engineering and simulation environment for education. It shall provide high causal, behavioral, workflow, and training-transfer fidelity inside a wholly fictional VirtualUniverse. It shall never communicate with, discover, configure, commission, download to, or operate physical industrial equipment. No adapter to a physical universe may exist.

CONSTRUCTION STATUS: This is the first authoring phase of one living directive. Unless Scott separately orders otherwise, Codex shall not begin product implementation from this incomplete directive.

## Phase 1 Contents



Later phases are intentionally absent from this revision. Reserved headings are not implementation requirements and shall not be inferred.

## Document Control

| Field | Controlling value |
|---|---|
| Final filename | PLC Engineering Simulator - Codex Master Implementation Directive.docx |
| Living-document rule | Phases 2-4 append to and revise this same document; do not create competing master directives |
| Highest product authority | Scott's explicit approved product decisions |
| Research authority | Govs PLC project.md, frozen at the hash above |
| Public product name | Not yet cleared; the title is a working descriptive name |
| Canonical educational term | Teacher Mode |
| Principal workflow reference | V21-era TIA-oriented training profile |
| Semantic foundation | Brand-neutral IEC 61131-3 concepts, independently implemented |
| Physical connectivity | Permanently excluded |
| Current authorization | Author Phase 1 only |

## Construction-Phase Ledger

| Authoring phase | Content | Status |
|---|---|---|
| Phase 1 | Product constitution, canonical vocabulary, scope, fidelity doctrine, immutable safety wall, clean-room policy, trust model, architectural invariants, requirement system, ambiguity rules, anti-placeholder policy, living-document governance | Authored in this revision |
| Phase 2 | Semantic kernel and virtual execution: project graph, persistence, fictional hardware/network, tags/types, OB/FC/FB/DB, LAD/FBD/SCL semantics, compiler, typed IR, runtime, commissioning, online/offline, watch/modify/force/trace, diagnostics | Reserved; not yet normative |
| Phase 3 | Engineering experience, HMI, virtual process, Engineering Mode, Learning Lens, Teacher Mode, scenarios, assessments, replay, libraries, accessibility, performance envelopes | Reserved; not yet normative |
| Phase 4 | Codex execution program, repository and milestone detail, full verification matrix, isolation proof, supply chain, release gates, transfer study, legal checklist, final Definition of Done and handoff | Reserved; not yet normative |

## How to Use This Directive

1. Read the authority hierarchy before interpreting any requirement.
2. Treat every requirement ID as stable. Never renumber or reuse one.
3. Treat the research report as evidence and context unless this directive adopts a point normatively.
4. Do not infer missing Phase 2-4 behavior from headings or from general knowledge.
5. If a decision falls under a mandatory stop rule, block only the affected work, record the decision request, and continue unaffected work.
6. Never trade away the safety wall, clean-room rules, or causal-fidelity doctrine for speed, convenience, visual similarity, or a demo.

7. A visible control, interface, mock, animation, or passing build does not prove a feature exists.

### Normative Keywords

| Keyword | Meaning |
|---|---|
| MUST / SHALL | Required. Violation blocks merge, release, or acceptance. |
| MUST NOT / SHALL NOT | Prohibited. Presence blocks merge, release, or acceptance. |
| SHOULD | Expected unless a documented ADR proves an equal or stronger result without changing product intent. |
| SHOULD NOT | Avoid unless a documented ADR proves necessity and preserves all higher requirements. |
| MAY | Optional and permitted only inside the approved scope. |
| BLOCKED | Work cannot proceed without an approved decision, additional evidence, or legal review. |
| DEFERRED | Intentionally assigned to a later authoring or implementation target. It is not implemented. |
| EXCLUDED | Permanently outside this product. It is not a future feature. |

# 1. Authority and Conflict Resolution

## 1.1 Authority hierarchy

**[PES-GOV-0001] MUST** interpret this project using the following order:

1. Applicable law, binding licenses, and the immutable product safety constraints in this directive form the outer boundary.
2. Scott's explicit, approved product decisions govern product intent.
3. This living Codex Master Implementation Directive governs what shall be built.
4. The frozen research report supplies technical, workflow, and risk evidence.
5. Approved decision records and ADRs govern implementation choices only within the authority left to them.
6. Code, tests, tickets, comments, mockups, and developer assumptions are subordinate to all items above.

**[PES-GOV-0002] MUST NOT** use a lower authority to weaken, reinterpret, or silently override a higher authority.

**[PES-GOV-0003] MUST** treat the research report's labels accurately:

- DOCUMENTED identifies publicly supported behavior or facts.
- INFERENCE identifies a reasoned conclusion, not a documented exact behavior.
- PROPOSED identifies simulator behavior recommended by the report.
- LEGAL INTERPRETATION is risk analysis, not legal advice.
- ENGINEERING RECOMMENDATION is an implementation or product judgment.

**[PES-GOV-0004] MUST** treat adopted requirements in this directive as normative regardless of the research label from which they originated. The source label remains attached for traceability and shall not be rewritten as stronger evidence.

**[PES-GOV-0005] MUST NOT** claim that the research report is a legal opinion, patent clearance, trademark clearance, freedom-to-operate analysis, or guarantee of legality.

## 1.2 Conflict protocol

**[PES-GOV-0006] MUST** create a BLOCKED decision record when two authorities appear to conflict. The record shall quote or precisely identify both statements, explain the conflict, list the affected requirement IDs and components, and state the minimum decision needed.

**[PES-GOV-0007] MUST NOT** resolve an authority conflict by selecting the easiest implementation, the closest vendor behavior, or the broadest feature scope.

**[PES-GOV-0008] MUST** construe ambiguity conservatively when physical capability, external communication, proprietary expression, user data, assessment integrity, or safety claims could be affected. Conservative means no new capability and no public claim until resolved.

**[PES-GOV-0009] MUST** treat any proposal to add physical industrial communication as a proposal for a different product. It cannot be approved by an ordinary ADR, feature flag, plugin, experimental branch, edition, or milestone inside this product.

## 1.3 Frozen research baseline

**[PES-GOV-0010] MUST** use the research report identified by filename and SHA-256 in Document Control as the Phase 1 baseline.

**[PES-GOV-0011] MUST NOT** expand scope automatically when a newer TIA release, IEC edition, browser capability, framework version, or industrial technology appears.

**[PES-GOV-0012] MUST** process new research through an evidence record and an approved change record before it changes a normative requirement.

**[PES-GOV-0013] MUST** replace fragile research citation tokens with stable evidence records containing source title, publisher, version/date, durable location, access date, and the claim supported. An unresolved source remains unresolved; Codex shall not invent bibliographic details.

# 2. Product Charter

## 2.1 Mission

**[PES-MSN-0001] MUST** build an original, professional PLC engineering and automation simulation environment for classroom learning, independent study, guided troubleshooting, and instructor-led assessment.

**[PES-MSN-0002] MUST** make the student perform a recognizable modern PLC engineering lifecycle:

1. Create or open a simulator-native project.
2. Add fictional virtual controllers and devices.
3. Configure virtual racks, modules, channels, addresses, logical networks, and topology.
4. Create tags, constants, data types, and data blocks.
5. create OB, FC, FB, instance DB, and global DB program structures.
6. Program in LAD, FBD, and SCL (Structured Text).
7. Compile genuine project, hardware, language, type, and dependency semantics.
8. Repair real inconsistencies and rebuild.
9. Start a fictional controller instance.
10. Review an internal Virtual Load Preview.
11. Perform an atomic Virtual Download to a VirtualControllerId.
12. Use RUN, STOP, monitoring, watch, modify, force, and trace semantics.
13. Operate a deterministic virtual process and virtual HMI.
14. Diagnose causal code, hardware, network-graph, process, and HMI faults.
15. Correct the underlying cause and verify the result.

**[PES-MSN-0003] MUST** make the engineering decisions and consequences authentic enough to transfer into an authorized laboratory workflow while keeping product identity, code, assets, devices, project formats, runtime, and communication capability original.

**[PES-MSN-0004] MUST NOT** be industrial-control software, a hardware configuration utility, a protocol client, a controller emulator, a firmware emulator, a TIA clone, or a Siemens-branded product.

**[PES-MSN-0005] MUST** define "fully functional" as fully functional inside VirtualUniverse. The phrase never implies physical compatibility, real controller deployment, vendor-project compatibility, safety certification, or industrial suitability.

## 2.2 Intended users and environments

**[PES-MSN-0006] SHOULD** serve mechatronics and industrial-automation students who lack continuous access to commercial engineering software or hardware.

**[PES-MSN-0007] MUST** operate in an offline classroom or home-study environment with no cloud service, remote font, CDN, telemetry service, license server, analytics endpoint, or internet connection required.

**[PES-MSN-0008] MUST** support a professional unassisted workflow for students and a separate explanatory/teaching experience without changing the underlying engineering semantics.

**[PES-MSN-0009] MUST** keep all claims educational. It shall not claim certification, equivalence, endorsement, production readiness, or suitability for real machine control.

## 2.3 Governing success definition

**[PES-ACC-0001] MUST** judge success by causal and workflow transfer: the same kinds of engineering decisions should produce the same kinds of engineering consequences.

**[PES-ACC-0002] MUST** count the product as failing if a polished UI masks fake compilation, canned diagnostics, a non-semantic editor, nondeterministic runtime, conflated offline/online state, or scenario-specific hard-coding.

**[PES-ACC-0003] MUST** count the product as failing if any production code path can address, enumerate, connect to, send to, load to, commission, or operate a physical industrial target.

**[PES-ACC-0004] MUST** target zero students leaving training with the belief that a simulator project can be loaded into a physical PLC.

# 3. Canonical Vocabulary, Modes, and Claims

## 3.1 Canonical product vocabulary

| Canonical term | Required meaning | Forbidden interpretation |
|---|---|---|
| VirtualUniverse | The complete in-memory/persisted fictional automation world | A wrapper around physical devices or host networking |
| VirtualNetwork | A domain graph of fictional interfaces, subnets, and links | TCP/IP, a host NIC, a local server, or industrial Ethernet |
| Virtual IP address | An inert validated domain value | A routable endpoint |
| VirtualControllerId | Opaque identity of a fictional running controller object | Hostname, IP, URL, port, socket, USB identity, or serial handle |
| Virtual Download | Atomic transfer of an internal build artifact to a VirtualControllerId | Physical PLC download or vendor load artifact |
| Virtual online session | Session and comparison with a virtual controller object | A device connection or automatic synchronization |

| Canonical term | Required meaning | Forbidden interpretation |
|---|---|---|
| InternalTagBus | Typed internal publication/subscription among simulator components | OPC UA, HTTP, WebSocket, localhost, or any external transport |
| Engineering Mode | Professional engineering experience without unsolicited teaching overlays | Simplified tutorial mode |
| Learning Lens | Optional read-only explanatory overlay on real state and execution | A separate fake runtime or answer generator |
| Teacher Mode | Course, lesson, fault, checkpoint, hint, grading, reset, and audit subsystem | Direct diagnostic/message injection or a cloud AI chatbot |

**[PES-VOC-0001] MUST** use **Teacher Mode** as the public and schema/API canonical name. "Instructor Mode" is a research-report alias only.

**[PES-VOC-0002] MUST** present the textual PLC language as **SCL (Structured Text)** on first use and may use **SCL** thereafter. The semantic foundation is independently implemented IEC-style Structured Text; the label does not claim Siemens compiler compatibility.

**[PES-VOC-0003] MUST** use fictional brand-neutral device categories such as Compact Controller, Modular Controller, Performance Controller, Technology Controller, Distributed I/O Station, Basic Operator Panel, Advanced Operator Panel, Variable-Speed Drive, and Servo Drive.

**[PES-VOC-0004] MUST NOT** use actual Siemens model numbers or marks as active catalog identities.

**[PES-VOC-0005] MUST NOT** use shorthand such as "clone TIA," "simulate an S7," "connect virtually to an IP," "download like the real target," or "fully compatible" in requirements, UI, documentation, tests, marketing, or code comments.

## 3.2 Product modes share one kernel

**[PES-EDU-0001] MUST** make Engineering Mode, Learning Lens, and Teacher Mode use one project model, compiler, diagnostic system, virtual runtime, process state, HMI state, and persistence system.

**[PES-EDU-0002] MUST NOT** implement a separate simplified compiler or runtime for lessons.

**[PES-EDU-0003] MUST** keep Learning Lens observational. It may pause, step, slow, inspect, annotate, and explain deterministic execution; it shall not alter program truth, compiler results, type rules, device state, or grading outcomes.

**[PES-EDU-0004] MUST** make Teacher Mode act through ordinary domain commands, scenario events, process faults, and virtual hardware faults.

**[PES-EDU-0005] MUST NOT** let Teacher Mode insert compiler errors, runtime diagnostics, HMI alarms, expected values, or "correct" program state directly.

**[PES-EDU-0006] MUST NOT** make Teacher Mode an online AI dependency. Any future AI-assisted feature requires a separate approved requirement set, privacy analysis, offline-capability decision, and safety-wall review.

### 3.3 Profile and version claims

**[PES-PROF-0001] MUST** use a V21-era workflow as the principal frozen training reference for the first complete product baseline.

**[PES-PROF-0002] MUST** implement a version-neutral semantic core and declarative TrainingProfile capability manifests.

**[PES-PROF-0003] MUST** keep project schema version independent from TrainingProfile version.

**[PES-PROF-0004] MUST** pin a project's selected profile and capability-manifest version. Opening or migrating a project shall not silently change runtime semantics.

**[PES-PROF-0005] MUST** treat V19-era and V20-era profiles as the first compatibility targets after the primary profile. Until their behavior is specified and tested, they shall be marked DEFERRED rather than presented as functioning profiles.

**[PES-PROF-0006] MUST NOT** claim exact controller-family fidelity for behavior the research report marks unresolved, including detailed OB priorities, recursion, optimized layouts, force edge cases, or vendor-specific SCL extensions.

## 4. Scope, Non-Goals, and Fidelity Doctrine

### 4.1 Required fidelity

**[PES-FID-0001] MUST** prioritize fidelity in this order:

1. Safety and physical isolation.
2. Correct domain semantics and causality.
3. Training-transfer workflow and state consequences.
4. Determinism, inspectability, and diagnostic navigation.
5. Professional interaction quality and accessibility.
6. Original visual polish.

**[PES-FID-0002] MUST** implement causal fidelity rather than screenshot fidelity.

**[PES-FID-0003] MUST** preserve meaningful distinctions that commercial engineering software teaches, including saved versus unsaved, built versus dirty, hardware build versus software build, offline source versus loaded artifact, loaded versus matching, RUN versus STOP, monitoring off versus on, raw process value versus CPU-visible value, modify versus force, initial versus actual versus retained value, and incoming versus cleared diagnostics.

**[PES-FID-0004] MUST** make failure arise from a domain invariant, parser, resolver, type checker, compiler rule, build state, runtime state, process state, HMI state, or explicit virtual fault.

**[PES-FID-0005] MUST** keep invalid structures editable where appropriate while preventing invalid executable output. An invalid LAD or FBD graph shall not produce executable IR.

**[PES-FID-0006] MUST** preserve stable identity through rename, maintain unresolved references through deletion, and restore original identity through undo.

**[PES-FID-0007] MUST** make diagnostics navigable to stable object identity and, when applicable, source range, graph node, pin, slot, channel, tag, or related object.

**[PES-FID-0008] MUST NOT** treat visual resemblance, screenshots, animated progress, or canned demo success as fidelity evidence.

## 4.2 Included product envelope

The completed four-phase directive will specify the following product envelope:

- Simulator-native project lifecycle and professional engineering workspace.
- Fictional controllers, modules, networks, topology, addresses, and process devices.
- Tags, types, data blocks, OB/FC/FB/DB organization, and stable references.
- Semantic LAD, FBD, and SCL.
- Dependency-aware compiler, original diagnostics, unified typed IR, and deterministic virtual PLC.
- Internal virtual commissioning, online/offline comparison, monitoring, watch, modify, force, and trace.
- Causal virtual hardware, network-graph, process, runtime, and HMI faults.
- Virtual process labs and deterministic scenario replay.
- Virtual HMI engineering and runtime through InternalTagBus.
- Engineering Mode, Learning Lens, Teacher Mode, assessments, audit, reset, and reusable scenarios.
- Simulator-native libraries, profiles, migrations, documentation, and classroom acceptance.

**[PES-SCP-0001] MUST** interpret this envelope as a promise to specify and implement genuine behavior in later phases, not authorization to create empty panels or placeholder APIs now.

## 4.3 Permanently excluded

**[PES-SCP-0002] MUST NOT** include any capability to communicate with or operate physical PLCs, HMIs, drives, remote I/O, gateways, instruments, sensors, actuators, robots, industrial networks, or host-connected devices.

**[PES-SCP-0003] MUST NOT** implement Siemens firmware, binaries, proprietary project formats, protocol payloads, device packages, engineering APIs, iconography, diagnostic prose, hardware illustrations, or vendor load artifacts.

**[PES-SCP-0004] MUST NOT** import or export a file intended to be accepted by a physical PLC, HMI, drive, vendor engineering system, or industrial communication tool.

**[PES-SCP-0005] MUST NOT** provide safety-rated programming, validation, certification, or claims. Safety objects, safety instruction sets, and safety certification simulation are excluded from Phases 1-4 unless Scott approves a separately researched legal/domain addendum. Ordinary educational interlocks shall be labeled non-safety-rated.

**[PES-SCP-0006] MUST NOT** provide remote collaboration, a cloud project server, telemetry, cloud grading, cloud AI, or a production local HTTP/WebSocket server.

### 4.4 Deferred and gated features

**[PES-SCP-0007] MAY** reserve architecture for generic PID, generic motion, generic drives, SFC, simulated local teamwork, recipes, user roles, source/version workflows, and advanced execution.

**[PES-SCP-0008] MUST** mark those features DEFERRED until later phases define semantics, risks, and acceptance tests.

**[PES-SCP-0009] MUST NOT** expose a deferred feature as an enabled control, working catalog item, selectable profile, successful command, or release claim.

**[PES-SCP-0010] MUST** require professional legal review before implementing behaviorally close auto-tuning, advanced motion trajectories, specialized drive models, unusual commissioning workflows, advanced digital-twin algorithms, or any Class 7 or Class 8 item.

## 5. Immutable VirtualUniverse Safety Wall

### 5.1 Constitutional invariant

> **VirtualUniverse has no adapter to PhysicalUniverse.**

**[PES-ISO-0001] MUST** enforce this statement as a permanent product-scope and security invariant.

**[PES-ISO-0002] MUST NOT** create a disabled adapter, generic PLC connection interface, transport provider, driver abstraction, protocol plugin, future-facing physical seam, feature flag, experimental connector, or "simulator now, hardware later" architecture.

**[PES-ISO-0003] MUST** model a controller session only by opaque VirtualControllerId and simulator state. The domain API shall contain no hostname, IP endpoint, URL, port, socket, interface index, MAC address used as a host target, USB identity, serial handle, Bluetooth identity, or generic connection string.

**[PES-ISO-0004] MUST** represent virtual addresses with opaque domain value types such as VirtualIpAddress. They may support parsing, formatting, subnet comparison, duplicate detection, and fictional topology logic. They shall not convert into host endpoint types.

**[PES-ISO-0005] MUST** implement fictional device discovery only as an in-memory query whose result is a subset of VirtualUniverse devices.

**[PES-ISO-0006] MUST** implement Virtual Download only as an atomic internal build-artifact transaction against a VirtualControllerId.

**[PES-ISO-0007] MUST** implement controller/process/HMI value exchange only through typed internal messages and InternalTagBus.

## 5.2 Forbidden communication capabilities

**[PES-ISO-0008] MUST NOT** contain or expose implementations of:

- S7, S7comm, or S7comm-plus;
- PROFINET DCP, PROFINET I/O, or PROFIBUS;
- EtherNet/IP or CIP;
- Modbus TCP or RTU;
- external OPC UA;
- EtherCAT, CAN, CANopen, DeviceNet, BACnet, MQTT, or other physical/industrial transports;
- vendor PLC, HMI, drive, or I/O SDKs;
- TIA Openness, Siemens engineering DLLs, or PLCSIM APIs;
- physical device discovery or host NIC enumeration;
- raw Ethernet, packet capture, or industrial protocol frames.

The list is illustrative. The category-wide ban controls renamed, wrapped, transitive, future, or equivalent capabilities.

**[PES-ISO-0009] MUST NOT** let shipped production code invoke or expose:

- TCP, UDP, raw sockets, TLS, DNS, HTTP, HTTPS, local HTTP, localhost servers, or generic socket APIs;
- fetch, XMLHttpRequest, WebSocket, WebRTC, EventSource to an endpoint, sendBeacon, WebTransport, or service-worker network interception;
- WebSerial, WebUSB, WebBluetooth, WebHID, WebNFC, WebMIDI, or later equivalent device APIs;
- serial ports, USB, Bluetooth, pcap, native device enumeration, or arbitrary filesystem devices;

- child-process execution, shell commands, dynamic library loading, native FFI, dlopen, arbitrary native bridges, or plugins able to reach those capabilities.

**[PES-ISO-0010] MUST** apply the prohibition to the entire shipped classroom product, including UI, project model, compiler, runtime, process engine, diagnostics, HMI, Teacher Mode, Learning Lens, importers, exporters, scripting, packaging glue, and production dependencies.

## 5.3 Narrow host allowlist

**[PES-SEC-0001] MAY** permit only these host capabilities in the production product:

- local rendering and user interaction;
- explicit user-initiated open/save of simulator-native project, archive, CSV, JSON, image, or report files approved by later requirements;
- controlled application-local persistence;
- typed UI-to-worker messaging;
- memory allocation;
- simulator-controlled monotonic virtual time inputs;
- printing or local document export only when a later requirement approves it and no external resource is loaded.

**[PES-SEC-0002] MUST** expose file persistence as bounded document operations, not arbitrary path traversal, executable launch, shell access, device files, or general-purpose host filesystem access.

**[PES-SEC-0003] MUST** ensure typed UI-to-worker IPC carries domain messages only. It shall not accept arbitrary code, URLs, shell strings, native method names, or generic transport descriptors.

## 5.4 Production versus development boundary

**[PES-SEC-0004] MAY** allow ordinary development and CI tools to download dependencies, run compilers, start test servers, and invoke child processes in the development environment.

**[PES-SEC-0005] MUST** keep those development capabilities outside production dependency graphs, shipped bundles, runtime permissions, and user-reachable code.

**[PES-SEC-0006] MUST** build the production classroom application without a local web server. Assets, workers, fonts, WASM, help, and examples shall be bundled and loaded locally without HTTP or WebSocket.

**[PES-SEC-0007] MUST** enforce a production Content Security Policy with at least **connect-src 'none'** and default-deny restrictions on external scripts, styles, fonts, images, media, objects, frames, forms, manifests, base-URI changes, and unsolicited navigation.

**[PES-SEC-0008] MUST NOT** include a network updater inside the trusted product. If a future updater is approved, it must be a separately packaged, separately permissioned product absent from classroom builds and unable to be invoked by trusted simulator code.

## 5.5 Threat and claim boundary

**[PES-SEC-0009] MUST** claim that the unmodified shipped product has no physical-industrial communication code path or capability. It shall not claim that a maliciously modified binary or compromised host operating system is metaphysically incapable of networking.

**[PES-SEC-0010] MUST** make zero-egress evidence process-scoped to the application and its child processes, while distinguishing unrelated host traffic.

**[PES-SEC-0011] MUST** fail release on attempted network syscalls or endpoint resolution even if a firewall blocks packets.

## 5.6 Untrusted files and scripting

**[PES-SEC-0012] MUST** treat every imported project, archive, CSV, JSON, library, scenario, image, or future script as untrusted input.

**[PES-SEC-0013] MUST** apply schema validation, canonical path validation, archive traversal prevention, duplicate-entry detection, compression-ratio limits, uncompressed-size limits, file-count limits, nesting limits, string/array/object limits, image-dimension limits, and deterministic resource budgets.

**[PES-SEC-0014] MUST NOT** execute code from a project, archive, library, scenario, HMI object, lesson, or sample.

**[PES-SEC-0015] MUST NOT** use eval, Function constructors, dynamic native modules, arbitrary JavaScript, arbitrary WebAssembly, macros, shell commands, or executable embedded content.

**[PES-SEC-0016] MUST** make any future HMI or assessment scripting a capability-limited original DSL or interpreter with deterministic execution, explicit resource limits, no host objects, no dynamic imports, no network, no filesystem, no process access, and no escape to general-purpose code.

## 5.7 Release-blocking isolation proof

**[PES-ISO-0011] MUST** make every isolation test release-blocking. Skipped, unavailable, flaky, or inconclusive isolation tests equal failure.

**[PES-ISO-0012] MUST** scan production dependency graphs, lockfiles, optional dependencies, aliases, native modules, dynamic imports, WASM imports, and packaged output for prohibited capabilities.

**[PES-ISO-0013] MUST** statically scan trusted and shipped source for forbidden browser, Node, native, FFI, subprocess, device, networking, and industrial APIs.

**[PES-ISO-0014] MUST** inspect every semantic/runtime WASM module. Allowed imports are limited to memory, deterministic typed host messaging, a controlled simulator clock, and narrowly controlled persistence. Networking, WASI sockets, arbitrary filesystem, process, and native FFI imports are forbidden.

**[PES-ISO-0015] MUST** run the complete product and course suite with all network adapters disabled or removed. No engineering, simulation, HMI, diagnostic, watch/force, Teacher Mode, Learning Lens, or assessment capability may be lost.

**[PES-ISO-0016] MUST** run zero-egress and zero-attempt tests covering project creation, virtual addresses, discovery, compilation, virtual load, RUN/STOP, HMI, monitoring, watch, modify, force, trace, diagnostics, faults, lessons, grading, save, and export.

**[PES-ISO-0017] MUST** fuzz all user-text and address-bearing fields with loopback, private, public, multicast, broadcast, IPv6, hostnames, URLs, industrial-looking ports, UNC paths, device paths, and malformed endpoint strings. All values shall remain inert data.

**[PES-ISO-0018] MUST** prove that device discovery results remain unchanged in the presence of a live LAN containing real or PLC-like devices.

**[PES-ISO-0019] MUST** prove at type, deserialization, reflection, and UI boundaries that Virtual Download accepts only VirtualControllerId.

**[PES-ISO-0020] MUST** prove every HMI binding resolves only through InternalTagBus.

**[PES-ISO-0021] MUST** prove exports contain no vendor project, firmware, load binary, deployable industrial payload, protocol frame, executable, or file directly accepted by a physical industrial tool.

**[PES-ISO-0022] MUST** retain machine-readable evidence for each isolation gate with artifact hash, test version, date, platform, result, and logs sufficient to reproduce the test.

# 6. Clean-Room, IP, Trademark, and Evidence Policy

## 6.1 Independent implementation

**[PES-CRM-0001] MUST** use original expression and independent implementation.

**[PES-CRM-0002] MUST** treat educational purpose as the mission, not permission to copy and not a substitute for legal analysis.

**[PES-CRM-0003] MAY** independently implement functional concepts such as compilation, project dependencies, scan execution, tag resolution, stateful blocks, hardware consistency, online/offline state, watch/force semantics, diagnostic navigation, contextual properties, and generic commands.

**[PES-CRM-0004] MUST NOT** copy Siemens screens, layout composition, icons, help prose, diagnostic prose or numbers, artwork, device illustrations, completion databases, project formats, compiler components, firmware behavior, or proprietary algorithms.

**[PES-CRM-0005] MUST** use original names, event codes, visual language, device identities, project structures, schemas, source representations, sample projects, and user documentation.

## 6.2 IP classification

Every externally inspired requirement shall be classified before implementation:

| Class | Meaning | Required disposition |
|---|---|---|
| 1 | Functional behavior | Independently implement |
| 2 | Industry or IEC convention | Implement from lawfully licensed standards or public behavior |
| 3 | Workflow behavior | Preserve useful workflow logic; redesign visuals and expression |
| 4 | Vendor-specific expression | Redesign |
| 5 | Branding or trademark | Replace or exclude |
| 6 | Proprietary technology | Create an original simulated equivalent |
| 7 | Patent or licensing concern | BLOCKED pending focused review |
| 8 | Uncertain or high-risk | BLOCKED pending professional legal review |
| 9 | Physical industrial communication | Permanently EXCLUDED |

**[PES-CRM-0006] MUST** default an unclassified or uncertain item to Class 8, not "probably permitted."

**[PES-CRM-0007] MUST NOT** begin implementation of a research-derived behavior until its requirement record contains an IP classification and disposition.

## 6.3 Permitted and forbidden sources

**[PES-CRM-0008] MAY** use:

- public Siemens documentation, SCE material, and public product/support pages as behavioral evidence;
- IEC descriptions or standards lawfully licensed to the team;
- public statutes and published judicial opinions;
- independent textbooks and tutorials for corroboration;
- independently created observations only under a written, counsel-approved observation protocol.

**[PES-CRM-0009] MUST NOT** use:

- Siemens source code, leaked code, leaked manuals, partner-only material, or confidential training material;
- decompiled or disassembled output;
- executable resources, extracted icons, resource packages, or memory scraping;

- protocol captures intended to reproduce vendor communications;
- encrypted project-format cracking;
- pirated software, license bypass, access-control circumvention, or API hooking;
- screenshots, manual diagrams, copied tables, copied hardware illustrations, or copied diagnostic text as implementation assets.

**[PES-CRM-0010] MUST** prohibit observation of an installed TIA Portal product for implementation verification until counsel reviews the applicable license terms and approves a written observation procedure.

**[PES-CRM-0011] MUST** keep screenshots and vendor assets out of production source, design files, tickets, code-generation prompts, training corpora, mockups, and asset pipelines unless counsel approves a quarantined evidence process. Quarantined evidence shall never be shipped.

## 6.4 Trademark, trade dress, and public language

**[PES-CRM-0012] MUST NOT** use Siemens, SIMATIC, TIA Portal, S7, WinCC, or PLCSIM marks as product identity, catalog identity, installer branding, repository branding, splash-screen branding, store listing, domain name, or implied affiliation.

**[PES-CRM-0013] MUST NOT** copy or sample a Siemens color system, icon silhouette family, device illustration style, typography, spacing system, screen composition, or overall trade dress.

**[PES-CRM-0014] MUST** hold every public comparative statement mentioning Siemens or TIA Portal as BLOCKED until trademark counsel approves the exact wording and notices.

**[PES-CRM-0015] MUST** treat the working title in this directive as descriptive only. It is not public-name clearance.

## 6.5 Evidence and contamination control

**[PES-CRM-0016] MUST** create CLEAN_ROOM_POLICY.md before feature implementation.

**[PES-CRM-0017] MUST** maintain a requirement evidence register with:

- requirement ID;
- paraphrased observed behavior;
- source title, publisher, version/date, durable location, and access date;
- report classification;
- IP class and disposition;
- simulator-owned implementation requirement;
- forbidden shortcut;

- author;
- reviewer;
- review status and date;
- implementation component;
- verification IDs.

**[PES-CRM-0018] MUST** quarantine a contribution suspected of contamination. It shall not enter builds, prompts, generated assets, or derived work until reviewed.

**[PES-CRM-0019] MUST** perform a clean rewrite without reusing tainted code, prose, assets, naming, layouts, or extracted structure when contamination is confirmed.

**[PES-CRM-0020] MUST** require contributor attestation that no forbidden source, asset, reverse-engineering output, protocol capture, or confidential material was used.

## 6.6 Asset and dependency provenance

**[PES-CRM-0021] MUST** register every shipped image, icon, font, sound, animation, template, sample project, translation, and other non-code asset with:

- asset ID;
- author/source;
- license and evidence location;
- created date;
- hash algorithm and original hash;
- derivative chain and modifications;
- generated-asset disclosure where applicable;
- reviewer, review status, and approval date.

**[PES-CRM-0022] MUST** reject unregistered or unapproved assets in CI.

**[PES-CRM-0023] MUST NOT** trace screenshots or icons, redraw vendor artwork, recolor vendor assets, or sample vendor branding as proof of originality.

**[PES-CRM-0024] MUST** generate an SBOM for release artifacts and review direct, transitive, optional, native, font, and asset licenses.

**[PES-CRM-0025] MUST** block dependencies whose license obligations are incompatible with the intended distribution or cannot be satisfied and documented.

### 6.7 Original native file boundary

**[PES-PRJ-0001] MUST** use only simulator-native, brand-neutral project and archive formats.

**[PES-PRJ-0002] MUST** use the provisional internal extensions **.vlabproj** for a project package and **.vlabarchive** for an archive until an approved product-name decision replaces them.

**[PES-PRJ-0003] MUST** make a project package a documented, versioned, non-executable container of canonical UTF-8 data and simulator-owned binary-neutral assets. A ZIP container is permitted only with the archive defenses in this directive.

**[PES-PRJ-0004] MUST** place a manifest in every project/archive containing schema version, pinned TrainingProfile ID and version, object-index version, required capabilities, file inventory, SHA-256 hashes, creation application version, and migration history.

**[PES-PRJ-0005] MUST** make project integrity failures visible and fail closed. It shall not silently discard unknown, corrupt, oversized, or hash-mismatched content.

**[PES-PRJ-0006] MUST NOT** use .apXX, .zapXX, a Siemens library format, PLCopen XML, vendor source export, or another real-tool format unless a later separately researched and legally approved directive explicitly adds a non-physical interoperability feature. No such approval exists in this revision.

**[PES-PRJ-0007] MUST** distinguish simulator-native CSV/JSON interchange from vendor or physical deployment. Native exports shall contain no executable code and shall be documented as simulator-only.

## 7. Security and Trust Model

### 7.1 Trust zones

| Zone | Contents | Trust rule |
|---|---|---|
| Trusted semantic core | project domain, type system, validator, compiler, typed IR, runtime, virtual hardware/network, process engine, diagnostics | Deterministic, capability-limited, no external communication |
| Trusted presentation | engineering UI, editors, Learning Lens, Teacher Mode UI, HMI renderer | Domain commands and typed messages only |
| Controlled persistence | project/archive open-save, application-local state, snapshots, recovery journal | Bounded document operations, validated data only |
| Untrusted content | imported projects, archives, CSV/JSON, images, future libraries/scenarios/scripts | Validate, limit, never execute |
| Development environment | package managers, compilers, test servers, CI tools | May use development capabilities but shall not enter production |
| Forbidden universe | physical equipment, host networking, industrial protocols, generic transports, device APIs, external services | No code path or adapter exists |

**[PES-SEC-0017] MUST** document the trust boundary in SECURITY_INVARIANTS.md and keep package ownership aligned to it.

**[PES-SEC-0018] MUST** keep persistence and presentation code from bypassing domain commands or writing trusted semantic state directly.

**[PES-SEC-0019] MUST** use explicit serialization schemas at trust boundaries. "Any," untagged arbitrary maps, dynamic class loading, and reflection-based invocation shall not cross into the semantic core.

**[PES-SEC-0020] MUST** validate message kind, schema version, payload size, object IDs, capability authorization, and state preconditions before a worker or core service executes a command.

## 7.2 Teacher/student data boundary

**[PES-TCH-0001] MUST** keep teacher-authored answer keys, hidden faults, checkpoints, and scoring rules logically separate from student-visible project state.

**[PES-TCH-0002] MUST** be honest that an offline local file cannot provide absolute secrecy against a student with full filesystem or process access. It shall provide role-appropriate UI separation, protected packaging where useful, integrity checking, and audit evidence without claiming cryptographic impossibility unless a later design proves it.

**[PES-TCH-0003] MUST** store student identity minimally. The default classroom model shall support local pseudonymous student IDs and shall not require names, email addresses, cloud accounts, or telemetry.

**[PES-TCH-0004] MUST** let teachers configure local audit-log retention and export. Later phases shall define exact defaults and privacy behavior before Teacher Mode is released.

**[PES-TCH-0005] MUST NOT** transmit grades, logs, project files, or identifiers outside the local product.

## 7.3 Security acceptance posture

**[PES-SEC-0021] MUST** fuzz every parser and deserializer that accepts untrusted content.

**[PES-SEC-0022] MUST** make parse, validation, and migration failures structured and recoverable. Catch-and-return-success is forbidden.

**[PES-SEC-0023] MUST** keep resource use deterministic or explicitly bounded. A project shall not allocate unbounded memory, unbounded recursion, unbounded event queues, or unbounded archive expansion.

**[PES-SEC-0024] MUST** disable recursion unless a later verified TrainingProfile explicitly enables and constrains it.

**[PES-SEC-0025] MUST** record security-relevant changes to CSP, package trust boundaries, import/export surfaces, worker IPC, file formats, or scripting in an ADR and threat-model update.

# 8. Constitutional Architecture Invariants

## 8.1 System topology

The required dependency direction is:

1. The Engineering UI issues typed commands to the Project Domain.
2. The Project Domain owns virtual hardware/network, PLC program, HMI engineering, library, process, and education objects.
3. Validators and the dependency engine analyze domain objects.
4. LAD, FBD, and SCL frontends produce semantic models.
5. The compiler lowers valid semantic models into one Unified Typed PLC IR.
6. One Virtual Controller Runtime executes that IR.
7. The runtime exchanges values with the deterministic virtual process.
8. Fault providers mutate virtual hardware/process state through commands.
9. Diagnostics arise from validation, build, runtime, hardware, process, or HMI state.
10. Monitoring, watch, modify, force, trace, HMI, Learning Lens, Teacher Mode, and assessment observe or act through defined internal contracts.
11. Project state, build artifacts, loaded runtime state, and snapshots persist as distinct objects.

**[PES-ARC-0001] MUST** enforce dependency direction. UI code shall not contain authoritative PLC semantics; runtime code shall not parse editor layout; Teacher Mode shall not bypass commands; persistence shall not manufacture valid state.

**[PES-ARC-0002] MUST** keep the semantic core platform-neutral and deterministic.

**[PES-ARC-0003] MUST** make every extension point domain-specific. No generic transport, generic executable plugin, generic native call, or arbitrary scripting extension is permitted.

## 8.2 Stable identity and project graph

**[PES-ARC-0004] MUST** represent every semantically referenceable project, hardware, network, language, HMI, library, process, lesson, scenario, assessment, diagnostic source, and runtime target object with an immutable UUID.

**[PES-ARC-0005] MUST** use RFC 9562 UUID version 4 by default for newly created objects. Display names, addresses, paths, array positions, block numbers, and source coordinates shall not serve as identity.

**[PES-ARC-0006] MUST** preserve UUID on rename, move, readdress, regroup, interface-compatible edit, and undo restoration.

**[PES-ARC-0007] MUST** create a new UUID for copy, template instantiation when independent, and imported objects intentionally duplicated as new objects.

**[PES-ARC-0008] MUST** retain a tombstone for deleted referenced objects for as long as a live reference, undo record, migration record, diagnostic, audit event, or snapshot requires it.

**[PES-ARC-0009] MUST** represent unresolved references explicitly with the target UUID and reference kind. Deletion shall not silently erase or retarget usages.

**[PES-ARC-0010] MUST** detect UUID collision on import. It shall reject ambiguous merge or perform an explicit, fully traced remap only when the import operation is defined to create independent objects.

**[PES-ARC-0011] MUST** maintain typed dependency edges and source/editor locations sufficient for where-used, caller/callee, type/DB usage, HMI binding, hardware-to-tag mapping, unresolved-reference filtering, and diagnostic navigation.

## 8.3 Command, transaction, event, and audit model

Every meaningful mutation shall be a domain command. The minimum conceptual result is:

```
DomainResult {
  success
  value?
  events[]
  diagnostics[]
  affectedObjectIds[]
  undoToken?
  beforeHash
  afterHash
}
```

**[PES-ARC-0012] MUST** route create, rename, delete, restore, move, copy, retype, bind, connect, disconnect, configure, compile request, load request, CPU state change, modify, force, fault, reset, lesson action, and migration through typed domain commands or explicitly read-only queries.

**[PES-ARC-0013] MUST** make commands atomic with respect to their declared transaction boundary. Failure shall leave either the previous valid state or a separately modeled unresolved/invalid engineering state, never a half-applied hidden mutation.

**[PES-ARC-0014] MUST** make undo/redo use command/event semantics and restore exact stable identity where the original object is restored.

**[PES-ARC-0015] MUST** record deterministic event ordering, affected object IDs, before/after hashes, and command provenance sufficient for crash recovery, replay, Teacher Mode audit, and testing.

**[PES-ARC-0016] MUST NOT** let UI components write domain objects directly or let a lesson mutate serialized files behind the domain model.

## 8.4 Canonical type system and semantic editors

**[PES-TYP-0001] MUST** create one canonical recursive type system shared by tags, DBs, block interfaces, LAD, FBD, SCL, addresses, runtime memory, watch/modify/force, trace, HMI bindings, assessment expressions, and profiles.

**[PES-TYP-0002] MUST** keep named-type identity distinct from structural shape and give type members stable identity.

**[PES-ARC-0017] MUST** represent LAD as a semantic graph/AST. Screen coordinates, wire routing, zoom, and element placement are layout metadata only.

**[PES-ARC-0018] MUST** represent FBD as a typed port graph with stable node, port, and edge identity plus explicit execution dependencies.

**[PES-ARC-0019] MUST** represent SCL with an independently implemented lexer, parser, AST, source ranges, scope resolver, control-flow model, and original language-service metadata.

**[PES-ARC-0020] MUST** share instruction definitions, type checking, conversions, call signatures, diagnostics, and runtime semantics across language frontends.

**[PES-ARC-0021] MUST NOT** execute LAD from screen coordinates, use a regex-only compiler, use eval for SCL, or maintain separate inconsistent runtimes for LAD, FBD, and SCL.

## 8.5 Unified typed IR and one runtime

**[PES-IR-0001] MUST** lower LAD, FBD, and SCL semantic models into one versioned, typed, serializable PLC IR.

**[PES-IR-0002] MUST** centralize arithmetic, conversions, comparisons, calls, timers, counters, storage access, monitor probes, source mappings, and error semantics in the shared compiler/runtime path.

**[PES-IR-0003] MUST** make build artifacts immutable and fingerprinted. A build shall identify project snapshot hash, compiler version, IR version, TrainingProfile ID/version, dependency closure, diagnostics, and source map.

**[PES-IR-0004] MUST NOT** produce a runnable artifact when a blocking error exists.

**[PES-IR-0005] MUST** reserve instrumentation points keyed by semantic node and source identity so monitoring, trace, Learning Lens, diagnostics, and assessment can observe one execution without changing it.

## 8.6 Deterministic virtual time, scheduling, and replay

**[PES-DET-0001] MUST** use simulator-controlled monotonic virtual time for the PLC scheduler, timers, counters with temporal behavior, process physics, trace, scenarios, lesson triggers, and assessment timing.

**[PES-DET-0002] MUST NOT** use wall-clock timers such as browser setTimeout as authoritative PLC or process time.

**[PES-DET-0003] MUST** define stable ordering for events sharing the same virtual timestamp and priority.

**[PES-DET-0004] MUST** include deterministic seed, event sequence, TrainingProfile hash, build hash, initial snapshot hash, simulator version, and scheduler version in replay identity.

**[PES-DET-0005] MUST** distinguish virtual timestamp from engineering-display wall-clock timestamp.

**[PES-DET-0006] MUST** guarantee that the same supported build, snapshot, profile, seed, and ordered events produce equivalent observable tag streams, outputs, diagnostics, trace data, HMI updates, and assessment results.

**[PES-DET-0007] MUST** reserve scan-start, input-sample, program-execution, output-commit, process-update, trace/diagnostic/HMI publication, and scan-end boundaries.

## 8.7 Separate state layers

**[PES-ARC-0022] MUST** keep these layers distinct:

- editable offline project source;
- saved project state;
- hardware build state;
- software build state;
- HMI build state;
- immutable build artifact;
- loaded virtual-controller artifact;
- current virtual runtime values;
- declared initial/start values;
- loaded baselines;
- retained values;
- raw virtual-process values;
- CPU-visible values;
- one-shot modifications;

- persistent force overlays;
- runtime snapshots;
- project/runtime equality or mismatch;
- monitoring active state.

**[PES-ARC-0023] MUST** model "go online" as a virtual session and comparison. It shall not automatically compile, download, synchronize, or equalize states.

**[PES-ARC-0024] MUST** model Virtual Download as preview plus explicit approval/cancellation and atomic commit/rollback.

**[PES-ARC-0025] MUST** make ForceRegistry global runtime state independent of any open table or pane.

## 8.8 Diagnostics and causal faults

The compiler diagnostic minimum is:

```
Diagnostic {
  code
  severity
  phase
  originalMessage
  objectId
  sourceRange?
  relatedObjectIds[]
  recoveryHint?
  buildId
}
```

The runtime/event diagnostic minimum is:

```
DiagnosticEvent {
  eventId
  code
  severity
  sourceObjectId
  virtualTimestamp
  engineeringTimestamp
  lifecycle
  originalTitle
  originalDetail
  relatedObjectIds[]
  relatedTagIds[]
  navigationTarget?
}
```

**[PES-DIA-0001] MUST** use original simulator codes and prose. It shall not copy vendor numbers or messages.

**[PES-DIA-0002] MUST** distinguish immutable build diagnostics from lifecycle-bearing runtime diagnostic events while permitting a unified UI.

**[PES-DIA-0003] MUST** derive diagnostics from ordinary validators, compiler rules, runtime transitions, device/process state, HMI consistency, persistence validation, or fault providers.

**[PES-DIA-0004] MUST** let Teacher Mode invoke commands such as RemoveModule, DisconnectVirtualLink, ChangeTagType, SetSensorFault, or SetActuatorFault and let ordinary engines derive the consequence.

**[PES-DIA-0005] MUST NOT** let a scenario, lesson, demo, or UI directly insert an expected compiler diagnostic, runtime fault, alarm, trace, monitored value, or passing assessment result.

**[PES-DIA-0006] MUST** retain navigation targets, related object identities, virtual timestamps, lifecycle correlation, and deterministic replay ordering.

### 8.9 Internal buses and future seams

**[PES-ARC-0026] MUST** define InternalTagBus as typed, quality-aware, timestamped internal publication/subscription. It shall operate by in-process calls or typed worker IPC, never localhost or network transport.

**[PES-ARC-0027] MUST** reserve typed domain registries for project object kinds, editors, properties, commands, validators, compilers, capability gates, navigation targets, fault providers, serializers, and migrations.

**[PES-ARC-0028] MUST** reserve safe schemas for scenario packages, lesson conditions, assessment expressions, snapshots, fault capabilities, and deterministic events without enabling arbitrary code.

**[PES-ARC-0029] MUST** allow later HMI, library, trace, technology object, source view, localization, and scenario types without replacing the canonical project graph.

**[PES-ARC-0030] MUST NOT** satisfy "reserved architecture" with empty buttons, no-op objects, placeholder transports, user-visible coming-soon panels, or generic interfaces that create forbidden capability.

## 9. Binding Technology and Repository Foundation

### 9.1 Adopted stack

**[PES-DEV-0001] SHOULD** use TypeScript and React for the engineering UI and application orchestration.

**[PES-DEV-0002] SHOULD** use a custom SVG/Canvas semantic renderer for LAD and FBD rather than an image-based or copied vendor renderer.

**[PES-DEV-0003] SHOULD** use a locally bundled Monaco-class editor component for SCL, with an original theme and independently generated language metadata.

**[PES-DEV-0004] MUST** implement the trusted project semantics, compiler, typed IR, scheduler, and PLC runtime in Rust compiled to capability-limited WebAssembly, unless Scott approves an ADR demonstrating an equally deterministic and more strongly isolated alternative.

**[PES-DEV-0005] MUST** execute virtual runtime/process work in isolated workers using typed messages so simulation cannot freeze the UI.

**[PES-DEV-0006] SHOULD** use a pnpm workspace for TypeScript packages and a Cargo workspace for Rust crates, with exact versions locked and release builds reproducible.

**[PES-DEV-0007] MUST** bundle all production dependencies, fonts, WASM, help, scenarios, and assets locally.

**[PES-DEV-0008] MUST** keep the trusted core free of OS networking, native FFI, arbitrary filesystem, shell, and device capabilities even if the desktop shell or browser engine internally supports them.

**[PES-DEV-0009] MUST** record the chosen desktop/classroom packaging model in a BLOCKED product decision before public artifact work begins. The decision shall define initial supported operating systems, installer versus portable delivery, local-file permissions, Chromium/WebView background networking controls, code signing, update separation, and offline verification.

## 9.2 Required top-level governance files

Before feature implementation, the repository shall contain:

```
CLEAN_ROOM_POLICY.md
SECURITY_INVARIANTS.md
LEGAL_REVIEW_CHECKLIST.md
CONTRIBUTOR_CLEAN_ROOM_ATTESTATION.md
THREAT_MODEL.md
REQUIREMENTS.md
IMPLEMENTATION_MATRIX.*
EVIDENCE_REGISTER.*
ASSET_PROVENANCE.*
DEPENDENCY_POLICY.*
OPEN_DECISIONS.md
RISK_REGISTER.md
CHANGELOG_DIRECTIVE.md
ADR/
  0001-no-physical-industrial-communication.md
  0002-original-project-format.md
  0003-unified-plc-ir.md
  0004-deterministic-virtual-time.md
```

**[PES-DOC-0001] MUST** create ADR-0001 with title "Physical Industrial Communication Is Permanently Out of Scope" and status "Project Safety Invariant."

**[PES-DOC-0002] MUST** state in ADR-0001 that physical capability cannot be added within this product through an ADR amendment. A physical-capable product requires a separate repository, authorization, legal analysis, threat model, and governance.

**[PES-DOC-0003] MUST** document original project format, unified IR, and deterministic virtual time before implementation depends on them.

**[PES-DOC-0004] MUST** keep evidence records and research notes separate from production assets.

## 9.3 Required package boundaries

The target repository shape is:

```
apps/
  engineering-ui/
packages/
  project-domain/
  virtual-hardware/
  virtual-network/
  plc-types/
  plc-block-model/
  lad-model/
  fbd-model/
  scl-frontend/
  validator/
  compiler/
  plc-ir/
  virtual-runtime/
  process-sim/
  diagnostics/
  monitor-force-trace/
  hmi-model/
  hmi-runtime/
  libraries/
  learning-lens/
  teacher-mode/
  assessment/
  persistence/
profiles/
scenarios/
assets/original/
docs/
tests/
  isolation/
```

**[PES-DEV-0010] MUST** treat this shape as a responsibility map, not permission to create empty packages for completion credit.

**[PES-DEV-0011] MAY** combine packages when doing so preserves trust boundaries, ownership, testability, and dependency direction. It shall record the reason in an ADR.

**[PES-DEV-0012] MUST NOT** create a network, transport, device-connector, vendor-adapter, protocol, external-HMI, remote-collaboration, or plugin-host package.

## 9.4 Baseline CI policy

**[PES-CI-0001] MUST** fail production merge or release when:

- a forbidden dependency or capability is added;
- a prohibited source API or WASM import appears;
- a remote asset, CDN, telemetry, analytics, or cloud dependency appears;
- an asset lacks provenance or approval;
- a vendor screenshot, logo, icon, device illustration, or copied prose enters production;
- an unclassified research-derived requirement enters implementation;
- a required test is skipped or flaky;
- determinism/replay diverges;
- migration loses identity or data;
- a lesson bypasses ordinary domain/diagnostic behavior;
- Virtual Download accepts any endpoint-like value;
- HMI uses any transport other than InternalTagBus;
- an exported artifact resembles or is accepted as a real industrial deployment artifact;
- traceability between a verified requirement and its tests is missing.

**[PES-CI-0002] MUST** scan the packaged artifact, not only source and lockfiles.

**[PES-CI-0003] MUST** produce an SBOM, license notice set, asset manifest, requirement-verification report, and isolation report for a release candidate.

# 10. Normative Requirement, Decision, and Change System

## 10.1 Stable identifiers

**[PES-REQ-0001] MUST** identify product requirements as **PES-AREA-NNNN**. The AREA identifies a stable domain; the number is non-semantic.

**[PES-REQ-0002] MUST NOT** encode authoring phase, software release, priority, status, or document section in a requirement ID.

**[PES-REQ-0003] MUST** identify supporting records separately:

| Record | Identifier |
|---|---|
| Source/evidence | SRC-NNNN |
| Architecture decision | ADR-NNNN |
| Product decision | DEC-NNNN |
| Open question | OQ-NNNN |
| Risk | RSK-NNNN |
| Change record | CR-NNNN |
| Verification case | VER-AREA-NNNN |

**[PES-REQ-0004] MUST** keep retired IDs as tombstones with a supersession or rejection reason. IDs shall never be recycled.

## 10.2 Atomic record schema

Every requirement record shall contain:

- stable ID;
- short title;
- normative keyword;
- one atomic, testable statement;
- rationale;
- scope/component;
- source pointer and research classification;
- IP class and disposition;
- dependencies;
- target software release or milestone;
- current truth state;
- positive acceptance condition;
- negative acceptance condition;
- verification IDs;
- ADR/decision/change links;
- owner and reviewer.

**[PES-REQ-0005] MUST** split compound requirements when one part could pass and another fail.

**[PES-REQ-0006] MUST** map every implemented behavior to at least one requirement and every MUST/MUST NOT requirement to positive or negative verification.

**[PES-REQ-0007] MUST** map every test to the requirements it verifies. Orphan tests and unverified requirements shall be visible in CI reports.

## 10.3 Truth states

| State | Meaning |
|---|---|
| NOT_STARTED | No implementation work |
| SCAFFOLDED | Internal structure only; not working and earns zero completion credit |
| PARTIAL | Some real behavior exists; acceptance is not met |
| IMPLEMENTED_UNVERIFIED | Intended behavior exists but required verification has not passed |

| State | Meaning |
|---|---|
| VERIFIED | All required positive, negative, isolation, persistence, and integration verification has passed |
| BLOCKED | Needs an approved decision, evidence, or legal review |
| DEFERRED | Intentionally assigned to a later target |
| EXCLUDED | Permanently outside the product |

**[PES-REQ-0008] MUST** use VERIFIED as the only state equivalent to complete.

**[PES-REQ-0009] MUST NOT** calculate percent complete from file count, package count, UI controls, lines of code, passing compilation, or SCAFFOLDED/PARTIAL items.

### 10.4 Change control

**[PES-GOV-0014] MUST** create a change record for any alteration to a Phase 1 requirement, authority rule, safety boundary, clean-room rule, canonical term, or architecture invariant.

**[PES-GOV-0015] MUST** include reason, affected IDs, research/evidence impact, security/IP impact, migration impact, test impact, decision authority, approval date, and supersession links.

**[PES-GOV-0016] MUST NOT** edit a controlling requirement only in code or an ADR. The directive and traceability records shall change first or in the same approved change.

## 11. Decision, Ambiguity, and Mandatory Stop Rules

### 11.1 Decisions Codex may make

**[PES-DEC-0001] MAY** let Codex decide an implementation detail without asking only when every plausible choice:

- is internal and reversible;
- preserves observable semantics and file compatibility;
- adds no network, device, native, process, plugin, cloud, AI, or credential capability;
- does not affect IP classification, branding, public claims, grading, teacher/student separation, privacy, or safety;
- stays within approved technology and dependency policy;
- can be objectively verified;
- satisfies all higher requirements.

Meaningful autonomous decisions shall still be recorded in an ADR or implementation note.

## 11.2 Mandatory stop categories

**[PES-DEC-0002] MUST** stop the affected work and ask Scott when a choice:

- touches physical/network capability or could weaken the VirtualUniverse wall;
- uses or resembles vendor assets, protocols, APIs, formats, names, model numbers, diagnostics, branding, or trade dress;
- changes public workflow semantics, TrainingProfile behavior, file format, migration, grading, Teacher Mode visibility, or student data handling;
- risks data loss, irreversible schema change, or backward incompatibility;
- requires cloud, credentials, telemetry, remote services, external AI, or an updater;
- adds eval, arbitrary scripting, FFI, child process, shell, native bridge, host device, generic transport, or executable plugin capability;
- is marked NEEDS MORE RESEARCH, Class 7, Class 8, or professional legal review;
- makes a safety, certification, compatibility, equivalence, endorsement, or production claim;
- conflicts with higher authority;
- cannot be verified objectively;
- would choose initial operating systems or the production packaging model;
- would expand scope beyond the authored phases.

**[PES-DEC-0003] MUST** stop and request additional verified research rather than invent exact controller-family OB numbers, priority/preemption matrices, nesting limits, recursion behavior, proprietary optimized DB layouts, vendor-specific conversions/built-ins, force edge cases, diagnostic numbers/prose, auto-tuning, complex motion, or legacy-language semantics.

## 11.3 BLOCKED decision record

Every blocked decision request shall contain:

```
Decision ID:
Affected requirement IDs:
Known facts:
Unknown or conflicting point:
Why Codex cannot decide safely:
Option A and impact:
Option B and impact:
Option C and impact, if useful:
Recommended option:
Exact approval or evidence needed:
Work that can continue:
```

**[PES-DEC-0004] MUST** bundle related questions so Scott receives the smallest coherent decision request.

**[PES-DEC-0005] MUST** continue unrelated work while the affected area remains blocked.

**[PES-DEC-0006] MUST NOT** treat silence, elapsed time, a placeholder, "close as possible," "educational," or an implementation guess as approval.

# 12. No-Fake-Completion and Anti-Placeholder Policy

## 12.1 Forbidden implementation theater

**[PES-QLT-0001] MUST NOT** count a feature as implemented because a pane, button, menu item, type, interface, package, schema field, animation, sample, or mocked path exists.

**[PES-QLT-0002] MUST NOT** ship:

- no-op commands;
- hard-coded success or catch-and-return-success;
- fake compile, load, online, scan, force, HMI, or diagnostic animations;
- canned errors or predetermined lesson results;
- sample-specific PLC/process logic in general engines;
- offline values displayed as monitored runtime values;
- HMI animation disconnected from InternalTagBus;
- mock/test doubles reachable in production;
- a regex-only compiler;
- SCL eval;
- LAD/FBD execution based on screen coordinates;
- hidden physical adapters disabled by configuration;
- scenario code that directly awards a pass;
- a "coming soon" control that appears operational.

**[PES-QLT-0003] MUST** fail closed when a feature is unavailable. The UI shall honestly disable or omit the action and identify the unmet capability without pretending success.

## 12.2 Permitted scaffolding

**[PES-QLT-0004] MAY** create scaffolding only when it:

- is not user-visible or release-reachable;
- contains no forbidden capability;
- fails closed;

- is labeled SCAFFOLDED in the implementation matrix;
- has an owner and removal/completion target;
- earns zero completion credit.

**[PES-QLT-0005] MUST NOT** create an abstract physical connection, generic transport, executable plugin host, network-capable HMI provider, or arbitrary scripting engine even as scaffolding.

## 12.3 Universal milestone Definition of Done

**[PES-QLT-0006] MUST** require every software milestone, when later authorized, to include:

- domain model and ownership;
- invariants;
- positive behavior;
- negative behavior;
- enumerated failure cases and recovery;
- stable identity and dependency behavior;
- persistence, migration, and undo where applicable;
- real UI integration where applicable;
- end-to-end workflow;
- deterministic unit/integration tests;
- property/fuzz/golden tests where applicable;
- isolation/security tests;
- clean-room evidence and asset provenance;
- documentation;
- requirement-to-test traceability;
- reproducible verification evidence.

**[PES-QLT-0007] MUST NOT** advance a milestone on screenshots, a successful build, a smoke test, a happy-path demo, or manual assertion alone.

**[PES-QLT-0008] MUST** keep a milestone open if any required test is skipped, flaky, unavailable, manually waived, or inconclusive.

# 13. Living-Directive Governance and Phase 1 Exit Gate

## 13.1 One document, four authoring phases

**[PES-GOV-0017] MUST** append and revise this same document for Phases 2-4 while preserving:

- exact filename;
- style system;
- requirement IDs;
- cross references;
- source hash history;
- change ledger;
- superseded requirement tombstones;
- open decisions and risk records.

**[PES-GOV-0018] MUST NOT** create separate competing master directives for later phases.

**[PES-GOV-0019] MUST** label unauthored later-phase material as reserved. It shall not create empty chapters that could be mistaken for complete requirements.

**[PES-GOV-0020] MUST** perform a cross-phase contradiction and coverage audit after every authoring phase.

## 13.2 Phase 1 acceptance checklist

Phase 1 is authored successfully only when:

- the authority hierarchy and conflict protocol are explicit;
- product mission, users, success definition, and "fully functional" boundary are explicit;
- canonical terminology resolves Teacher Mode and SCL/Structured Text naming;
- scope, exclusions, deferrals, and claims are explicit;
- the safety wall is category-wide, product-wide, immutable, and testable;
- production/development capability boundaries and the narrow host allowlist are explicit;
- untrusted file and scripting rules are explicit;
- clean-room, classification, provenance, contamination, trademark, and legal gates are explicit;
- stable identity, command/event model, typed IR, deterministic time, state separation, diagnostics, InternalTagBus, and extension rules are constitutional invariants;
- stack defaults, packaging decision gate, governance files, package boundaries, and baseline CI rules are explicit;

- requirement IDs, truth states, change control, stop rules, and anti-placeholder rules are explicit;
- later phases are partitioned without pretending their details are authored;
- the source report identity and hash are recorded;
- the DOCX passes structural and visual QA.

**[PES-ACC-0005] MUST** mark this revision "Phase 1 authored; Phases 2-4 not yet authored."

**[PES-ACC-0006] MUST NOT** treat completion of Phase 1 authoring as completion of the master directive.

**[PES-ACC-0007] MUST NOT** authorize product coding from this incomplete directive unless Scott separately gives explicit implementation authorization before Phases 2-4 are complete.

## 13.3 Open decisions carried forward

| ID | Decision | Why open | Required no later than |
|---|---|---|---|
| OQ-0001 | Initial supported operating systems and packaged classroom shell | Platform-neutral core does not choose a deployable artifact | Before product packaging work |
| OQ-0002 | Final public product name, logo, and compatibility language | Requires original design and trademark review | Before public branding |
| OQ-0003 | Concrete fictional controller/module profile values | Research defines the model, not exact generic limits | Phase 2 hardware specification |
| OQ-0004 | Identifier grammar, case rules, scope/shadowing, and automatic address allocation | Observable semantics require precise rules | Phase 2 tag/type specification |
| OQ-0005 | Exact project object schema and migration/downgrade policy | Phase 1 fixes boundaries but not the full domain schema | Phase 2 persistence specification |
| OQ-0006 | LAD/FBD legality corpus and vendor-specific SCL edge behavior | Research flags unresolved fidelity details | Phase 2; unsupported details remain blocked |
| OQ-0007 | Teacher/student file protection and local audit retention defaults | Needs UX/privacy decisions | Phase 3 Teacher Mode specification |
| OQ-0008 | Accessibility conformance target and performance/capacity budgets | Must be objective before experience acceptance | Phase 3 |
| OQ-0009 | PID, motion, SFC, classic HMI, safety-awareness, and legacy-language scope | Deferred and partly legal/research-gated | Phase 4 or separate addendum |
| OQ-0010 | Human training-transfer study design and authorized lab baseline | Needs course/instructor context | Phase 4 |

## 13.4 Risks carried forward

| ID | Risk | Phase 1 control |
|---|---|---|
| RSK-0001 | A "virtual network" abstraction accidentally becomes host networking | Opaque value types, no endpoint API, product-wide capability ban, zero-egress proof |
| RSK-0002 | UI fidelity drifts into copying or trade dress | Original expression, evidence classification, asset provenance, counsel-gated public comparisons |
| RSK-0003 | Lessons become canned demonstrations | Command-based causality and explicit prohibition on injected messages/results |
| RSK-0004 | Separate language runtimes diverge | Unified typed IR and one runtime |
| RSK-0005 | Time and replay become nondeterministic | Simulator clock, event ordering, profile/build/snapshot hashes |
| RSK-0006 | Untrusted projects become an execution or resource attack | No code execution, strict schemas, archive limits, fuzzing |
| RSK-0007 | Scaffolding is presented as progress | Truth states, zero credit for scaffolding, milestone Definition of Done |
| RSK-0008 | New research silently moves the target | Frozen baseline and approved evidence/change records |
| RSK-0009 | Safety education is mistaken for safety engineering | No safety-rated claims or feature set without separate addendum |
| RSK-0010 | Browser or desktop shell background activity violates zero egress | Packaging decision gate, bundled assets, CSP, process-scoped syscall/packet tests |

# Appendix A. Canonical Glossary

| Term | Definition |
|---|---|
| Build artifact | Immutable, fingerprinted, simulator-owned executable representation produced only by a successful compiler pipeline |
| Causal fidelity | Underlying state and rules produce observable consequences; UI or lesson code does not manufacture the outcome |
| Clean room | Independent implementation from permitted evidence with provenance, classification, original expression, and contamination controls |
| Engineering timestamp | Human-facing wall-clock metadata; never authoritative simulation time |
| Fictional device | Original controller, module, HMI, drive, or process identity existing only inside VirtualUniverse |
| InternalTagBus | Typed internal value/quality/time distribution between simulator components |
| Learning Lens | Optional read-only explanatory instrumentation over the real kernel |
| PhysicalUniverse | Physical PLCs, HMIs, drives, I/O, devices, networks, host endpoints, and industrial protocols; permanently unreachable |
| Profile | Versioned declarative capability manifest controlling simulator semantics; independent from project schema |
| Project schema | Versioned representation of simulator-native objects and files |
| Teacher Mode | Local lesson, fault, checkpoint, hint, grading, reset, and audit system operating through real domain state |
| Training-transfer fidelity | Ability to recognize and apply comparable engineering workflow and consequences in an authorized lab without copying vendor expression |
| VirtualUniverse | Sole addressable automation universe of the product |
| Virtual Download | Internal atomic deployment of a simulator build to a VirtualControllerId |
| Virtual time | Deterministic simulator-controlled time used by PLC execution, process physics, scenarios, trace, and assessment |

# Appendix B. Allowed and Forbidden Capability Summary

| Allowed inside production | Forbidden inside production |
|---|---|
| Typed domain commands and queries | Generic transports or physical adapters |
| In-memory VirtualNetwork graph | Host NIC enumeration, sockets, DNS, HTTP, localhost |
| VirtualControllerId | Hostname, URL, port, socket, USB/serial handle |
| InternalTagBus and typed worker IPC | OPC UA, WebSocket, fieldbus, protocol SDKs |
| User-initiated bounded project open/save | Arbitrary filesystem, device files, process execution |
| Simulator-controlled virtual clock | Wall-clock timer as PLC/process authority |
| Locally bundled assets and help | CDN, telemetry, cloud, remote fonts, update checker |
| Simulator-native non-executable import/export | Vendor project/load artifacts or protocol payloads |
| Capability-limited declarative DSL, if later approved | eval, arbitrary JS/WASM, macros, plugins, native FFI |

# Appendix C. Requirement Record Template

```
ID:
Title:
Keyword:
Atomic requirement:
Rationale:
```

```
Scope/component:
Source ID and location:
Research classification:
IP class and disposition:
Dependencies:
Target release/milestone:
Truth state:
Positive acceptance:
Negative acceptance:
Verification IDs:
ADR/decision/change links:
Owner:
Reviewer:
```

## Appendix D. Decision Request Template

```
Decision ID:
Date opened:
Owner:
Affected requirement IDs:
Known facts and evidence:
Unknown or conflict:
Why this cannot be decided autonomously:
Option A:
Option B:
Option C, if useful:
Recommendation:
Security/IP/data/migration/UX impacts:
Exact approval or evidence needed:
Unblocked work that will continue:
Resolution:
Approval and date:
```

## Appendix E. Phase 1 Source Crosswalk

| Directive area | Research report anchor |
|---|---|
| Mission, modes, version baseline, assumptions | Executive Summary |
| Professional workflow and fictional product language | Product Workflow, Version Baseline, and Complete Feature Taxonomy |
| Stable identity, libraries, hardware, virtual networks, tags | Project lifecycle through Tags and addressing |
| Shared language/compiler/runtime contracts | Program structure through Cross references and navigation |
| Causal failure, UI interaction, Learning Lens, Teacher Mode | Failure Modes, UI Fidelity, Teacher System, and Classroom Scenarios |
| Clean room, trademark, reverse engineering, provenance | Intellectual Property, Legal Boundaries, Clean-Room Process, and Simulation Wall |
| Physical-isolation tests and future-proofing | Runtime technology wall through Future-proofing |
| Architecture, command model, IR, stack | Architecture, Master Feature Matrix, and Coverage Analysis |
| Milestone truth, tests, acceptance, stop/research questions | Codex Implementation Roadmap, Verification Plan, and Final Handoff |

| Directive area | Research report anchor |
|---|---|
| Binding MUST/MUST NOT rules | Final Codex marching orders |
| Repository and release gates | Suggested repository layout through Final acceptance standard |

## Appendix F. Phase 1 "Do Not Build This" Register

The following are EXCLUDED, not backlog items:

- a TIA Portal clone or pixel copy;
- Siemens/SIMATIC/S7/WinCC/PLCSIM product or device identities;
- Siemens screenshots, icons, artwork, manuals, help text, diagnostics, event IDs, firmware, binaries, DLLs, APIs, or project formats;
- any physical PLC/HMI/drive/I/O connection, discovery, enumeration, commissioning, download, upload, monitoring, control, or protocol;
- S7, PROFINET, PROFIBUS, EtherNet/IP, CIP, Modbus transport, external OPC UA, EtherCAT, CAN/CANopen, DeviceNet, BACnet, MQTT, or vendor SDK communication;
- host NIC selection, raw sockets, DNS, HTTP, localhost server, WebSocket, WebRTC, WebSerial, WebUSB, WebBluetooth, WebHID, native FFI, or child-process capability;
- a hidden, disabled, premium, experimental, or future physical adapter;
- real-tool import/export or real load artifacts;
- a cloud-required runtime, cloud grading, telemetry, analytics, remote font, CDN, or embedded updater;
- SCL eval, LAD coordinate execution, regex-only compilation, multiple inconsistent language runtimes;
- canned compile/load/diagnostic/lesson behavior;
- Teacher Mode directly inserting expected messages or results;
- user-visible placeholder features represented as implemented;
- arbitrary executable plugins, macros, project scripts, or HMI code;
- safety-rated claims or a simulated safety engineering product;
- public compatibility or affiliation claims not approved by counsel.

> **Phase 1 closing rule:** The foundation is now explicit. The product may become broad, realistic, polished, and deeply functional only inside these boundaries. No later phase may buy fidelity by weakening originality, determinism, causal behavior, or physical isolation.